WINTER IN DATA SCIENCE 2025

Market Mood and Moves: Sentiment-Driven Stock Prediction

END TERM REPORT

Introduction

The Market Mood and Moves project aims to study the relationship between financial market movements and the underlying sentiment expressed in news and textual data. Financial markets are not driven purely by numerical fundamentals; investor psychology, expectations and reactions to news play a significant role in shaping short-term and medium-term price movements.

Note- This report outlines the **theoretical understanding, design choices, and learning outcomes** from Weeks 1, 2 and 3. All implementation details and code files are provided separately in the GitHub repository.

Week 1: Market Data and Text Processing Foundations

1 Understanding Financial Market Data

The first step involved understanding the structure and relevance of financial market data. Equity price data typically consists of Open, High, Low, Close, and Volume (OHLCV) values, which capture different aspects of intraday and interday price behaviour.

Key concepts studied include:

- **Returns vs. prices**: Returns are more suitable for statistical analysis as they are stationary compared to raw prices.
- **Market efficiency and information flow**: News and information are reflected in prices with varying delays depending on market conditions.
- **Time alignment issues**: Financial data is time-indexed, making synchronization with textual data a non-trivial task.

2 Motivation for Textual Analysis in Finance

Modern financial markets are highly sensitive to information disseminated through news articles, corporate announcements, analyst reports, social media, online discussions, etc.

While numerical indicators capture what happened in the market, textual data often explains why it happened. This motivates the use of NLP techniques to extract sentiment and contextual signals from unstructured text.

Independent reading focused on:

- Behavioural finance theories
- The role of sentiment in asset pricing
- Differences between general sentiment and finance-specific sentiment

This clarified why generic sentiment tools may be insufficient for financial applications.

3 Text Preprocessing and Linguistic Normalization

Raw text data is noisy and unsuitable for direct analysis. Therefore, a detailed study of text preprocessing techniques is needed.

Key preprocessing steps include:

- **Tokenization**: Breaking text into individual words or tokens.
- **Stop word removal**: Eliminating common words that carry little semantic meaning.
- **Lowercasing and normalization**: Ensuring uniform representation.
- **Lemmatization**: Reducing words to their base or dictionary form.

Understanding these steps from a linguistic perspective was essential to appreciate how preprocessing choices influence downstream sentiment analysis. The preprocessing pipeline was tested on sample financial headlines to validate correctness.

4 Lexicon-Based Sentiment Analysis (VADER)

Lexicon-based sentiment analysis was explored as a baseline approach. VADER (Valence Aware Dictionary and sEntiment Reasoner) assigns sentiment scores based on predefined word dictionaries and heuristic rules.

Key learnings:

- VADER is lightweight and interpretable.
- It performs well on short, informal text.
- However, it lacks domain-specific understanding of financial terminology.

For example, words like “liability” or “volatile” may carry neutral or technical meanings in finance but are treated negatively in general-purpose lexicons. This highlighted the need for more specialized models.

Week 2: Contextual Language Models and Financial NLP

1. Why Static Word Embeddings Fail for the Word "Bank" and the Need for BERT

Traditional static word embeddings such as Word2Vec or GloVe assign a single fixed vector to each word, irrespective of the context in which the word appears. This approach fails for words with multiple meanings.

For example, the word "bank" can appear in different contexts:

- "He deposited money in the bank." → Here, bank refers to a financial institution
- "The river bank was flooded." → Here, banks refers to a geographical landform

In static embeddings, both meanings share the same vector representation, leading to semantic ambiguity. As a result, downstream tasks like sentiment analysis or financial text classification suffer from misinterpretation.

Importance of BERT Architecture:

BERT (Bidirectional Encoder Representations from Transformers) overcomes this limitation by generating contextual embeddings. Instead of assigning a fixed vector to a word, it reads the entire sentence bidirectionally and generates word representations conditioned on surrounding words.

Thus, the embedding for "bank" dynamically changes based on context, allowing accurate semantic understanding which is a critical requirement in financial text analysis, where subtle wording can change interpretation by a lot.

2. Components of BERT Input Embedding

Each token fed into BERT is represented as the sum of three embeddings:

[Token Embedding] + [Segment Embedding] + [Position Embedding]

(a) Token Embeddings represent the actual tokens obtained after WordPiece tokenization

For example: "unbelievable" → "un", "believable"

(b) Segment (Sentence) Embeddings are used to distinguish between Sentence A and Sentence B. It is important for tasks like Next Sentence Prediction

- Tokens in Sentence A → Segment ID 0
- Tokens in Sentence B → Segment ID 1

(c) Positional Embeddings

- Since Transformers lack inherent word order awareness, positional embeddings encode the position of each token in the sequence and also, preserve syntactic structure and word order

The combination of these three embeddings allows BERT to capture semantic meaning, sentence relationships and word order simultaneously.

3. The 80–10–10 Masking Rule in BERT

BERT is trained using Masked Language Modeling (MLM), where 15% of tokens in a sentence are selected for prediction.

Among these selected tokens, 80% are replaced with the [MASK] token, 10% are replaced with a random word and 10% remain unchanged

Why this rule matters?

- It prevents the model from becoming overly dependent on the [MASK] token
- It improves robustness during real-world inference where [MASK] is absent
- It encourages learning deeper contextual representations

This strategy helps BERT generalize better across unseen text.

4. FinBERT: Domain-Specific Adaptation for Financial Text

4.1 Three-Stage Training Pipeline of FinBERT

FinBERT extends BERT by adapting it specifically for the financial domain using a three-stage pipeline:

Stage 1: General Language Pre-training- Here, model is initialized using BERT-base. It is then trained on large general-domain corpora (Wikipedia + BookCorpus) where it learns grammar, syntax and general language structure.

Stage 2: Domain-Adaptive Pre-training- The model is further pre-trained on financial text corpora which helps the model understand financial terminology, jargon and writing style.

For example: “bullish outlook”, “earnings surprise”, “credit downgrade”

Stage 3: Task-Specific Fine-tuning- Model is fine-tuned for specific tasks such as financial sentiment analysis and market news classification for which it uses labelled financial datasets

5. Domain Adaptation in NLP

Domain adaptation refers to the process of transferring a pre-trained language model to a specific domain by exposing it to domain-relevant data.

In the context of financial NLP, general BERT understands everyday language but lacks understanding of financial semantics. Domain adaptation bridges this gap by aligning word

meanings with financial usage and reducing semantic mismatch between training and target data. This step is crucial because financial language differs significantly from general English in tone, vocabulary and structure.

6. Why Specific Datasets Are Used in FinBERT

6.1 Why TRC2-Financial for Pre-training?

TRC2-Financial is a large-scale corpus consisting of financial news articles, market reports and corporate disclosures.

It is ideal for domain-adaptive pre-training because it provides unlabeled but domain-rich data, covers diverse financial contexts and helps the model internalize financial semantics at scale

This stage improves the model's understanding of financial language before any supervised learning.

6.2 Why Financial PhraseBank for Fine-tuning?

Financial PhraseBank is a manually annotated dataset where sentences are labeled as one from positive, negative or neutral.

It is used for fine-tuning because it provides high-quality labelled data, focuses on sentence-level financial sentiment and enables supervised learning for precise classification.

Fine-tuning on this dataset aligns the model's representations with sentiment-oriented financial tasks, such as analyzing market mood from news headlines.

Week 3: Dimension of Time and Conclusions

The third week of the project made a transition from static analysis to dynamic modelling. While Weeks 1 and 2 focused on understanding financial data and extracting sentiment from text using FinBERT, Week 3 addressed a fundamental limitation of traditional models: time dependence. Financial prices evolve as sequences, where current values depend strongly on historical context.

1. Probabilistic View of Financial Time Series

Financial markets violate the common I.I.D. (Independent and Identically Distributed) assumption used in many machine learning tasks. Instead, stock prices form a dependent sequence where the price at time *t* is influenced by previous prices.

Using the chain rule of probability, stock prices were formalized as a sequence where the joint probability is factorized into conditional probabilities. Since conditioning on the entire history is infeasible, two approximation strategies were studied:

- **Markov-style windowed models**

Markov-style windowed models approximate the full history by assuming that the current value depends only on a **fixed number of most recent observations**. This is inspired by the **Markov assumption**, which states that the future is conditionally independent of the past given the present. Mathematically: $P(x_t \mid x_1, \dots, x_{t-1}) \approx P(x_t \mid x_{t-\tau}, \dots, x_{t-1})$ where τ is the window length.

A sliding window of the last τtime steps is constructed. This window is flattened into a feature vector. A standard regression or classification model (e.g., Linear Regression, MLP, Random Forest) is trained to predict x_t. For example, using the last 30 days of returns to predict tomorrow's return.

- **Latent variable models**

Latent variable models replace explicit historical windows with an **internal hidden state** that summarizes all relevant past information. Instead of storing raw past values, the model maintains a learned representation of history.

Mathematically,

$$h_t = f(x_t, h_{t-1})$$
$$P(x_t \mid x_1, \dots, x_{t-1}) \approx P(x_t \mid h_{t-1})$$

Here, $h_t$is the **latent (hidden) state** and $f(\cdot)$is a learnable function (e.g., an RNN or LSTM).

The hidden state acts as a **compressed memory** of the entire past. There is no explicit limit on how far back the model can remember as the memory is controlled by the learning dynamics rather than a fixed window.

So, the crux is that **windowed models** answer *"What happened in the last 30 days?"* while the **latent models** answer *"What is the current market state based on everything that has happened so far?"*

2. Recurrent Neural Networks (RNNs)

Recurrent Neural Networks were introduced as a direct implementation of latent variable sequence models. Unlike feedforward networks, RNNs maintain a **hidden state** that evolves over time and carries information from previous steps.

Key learnings:

- How recurrent connections allow information persistence across time steps.

A recurrent connection is a feedback loop in which the hidden state from the previous time step is fed back into the network along with the current input. This creates a form of memory.

At each time step *t*, the network receives the current input x_t and the hidden state from the previous step h_{t-1}.

This feedback mechanism allows the network to **carry forward information from the past**, enabling it to learn temporal patterns such as trends, momentum, or regime changes in financial markets.

It is important as because of recurrent connections, the network remembers past information without explicitly storing the entire history. Patterns like "a steady uptrend over the last 10 days" can influence current predictions. The model can learn dependencies that span multiple time steps.

In finance, this persistence allows the model to encode concepts such as market volatility, trend direction and reaction delays to news events.

- The formulation of hidden state updates using both current input and past memory.

The hidden state is the internal memory of an RNN. It summarizes all relevant information from the past that the network has learned to retain.

At time step *t*, the hidden state is updated as:

$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b)$$

Where:

x_t: Input at time *t* (e.g., price return, sentiment score)

h_{t-1}: Hidden state from the previous time step

W_{xh}: Weight matrix for the input

W_{hh}: Weight matrix for the recurrent (memory) connection

b: Bias term

ϕ: Activation function (typically tanh or ReLU)

This equation shows that the new hidden state is influenced by current input which captures new information entering the system and past hidden state that preserves relevant historical context. The activation function ensures non-linearity, allowing the model to learn complex temporal relationships.

The hidden state acts as a compressed summary of the past, not a raw record. The network learns *what* to remember and *what* to forget through training.

For example, if recent sentiment is strongly negative, the hidden state may encode "bearish market mood". If prices have been stable for a long period, the hidden state may represent "low volatility regime".

This dynamic update mechanism allows RNNs to continuously adapt their memory based on evolving market conditions.

- The concept of Backpropagation Through Time (BPTT) for training sequence models.

Training an RNN is more complex than training a feedforward network because the same parameters are used at every time step and the output at a given time depends on previous computations.

What is Backpropagation Through Time?

Backpropagation Through Time (BPTT) is an extension of standard backpropagation designed for sequence models.

The idea is to u**nroll the RNN across time steps**, treating each time step as a layer in a deep network, compute the loss at the output and propagate gradients backward through each time step to update the shared parameters.

"through time" means errors at the final time step depend on computations performed at earlier time steps. BPTT propagates these errors backward along the temporal dimension.

For example, a wrong prediction today may be due to incorrect memory formation several days ago. BPTT assigns responsibility for this error to earlier hidden states and updates weights accordingly.

However, there are two challenges in this because the gradient is computed as a product of many Jacobian matrices.

1. **Vanishing Gradients:** Gradients shrink exponentially as they are propagated backward and hence, the model fails to learn long-term dependencies.
2. **Exploding Gradients**: Gradients grow uncontrollably and hence, training becomes unstable.

These issues explain why standard RNNs struggle with long sequences and motivate the use of LSTMs and gradient clipping.

BPTT allows the model to learn how events from the past influence future prices, market reactions unfold over time and delayed effects of news and sentiment shape price trends.

A major limitation in RNNs identified was the vanishing and exploding gradient problem, which restricts standard RNNs from learning long-term dependencies. This limitation is particularly problematic in finance, where events from weeks ago may still influence current market behaviour.

3. Long Short-Term Memory (LSTM) Networks

To overcome the shortcomings of RNNs, the Long Short-Term Memory (LSTM) architecture was studied. LSTMs introduce a dedicated **cell state** to store long-term information and use gating mechanisms to control information flow.

The following components:

- **Forget Gate**- decides which historical information is no longer relevant.
- **Input Gate**- controls how much new information should be stored.
- **Output Gate**- determines what information is exposed as the hidden state.

By separating long-term memory from short-term output, LSTMs enable stable gradient flow and effective learning over long sequences. This makes them well-suited for modelling financial time series with regime shifts, trends and delayed reactions to news.

4. Multimodal Market Model Design

Multimodal time-series model combines:

1. **Numerical Data** – Historical OHLCV data and derived technical indicators.
2. **Semantic Data** – Daily sentiment scores obtained from FinBERT.

5. Feature Engineering

To improve model performance and stability, raw prices were transformed into stationary representations such as log returns. Additional indicators like RSI, MACD, and rolling volatility were incorporated to provide trend and momentum context.

Multimodal Input Structure

Each training sample consisted of a sliding window of historical data, forming a 3D tensor of shape:

(Batch Size, Sequence Length, Number of Features)

By integrating sentiment as an explicit feature, the LSTM was exposed not only to what the market did, but also to why it may have behaved that way. This enabled the model to learn leading relationships between sentiment shifts and future price movements.

6. Implementation Pipeline

Week 3 emphasized clean and modular implementation using PyTorch:

- A custom **Dataset class** was designed to handle sliding-window sequence generation.
- A **Sentiment-Aware LSTM model** was implemented using a many-to-one architecture.
- A complete **training and evaluation pipeline** was constructed, including gradient clipping for stability.

Visualization of predictions versus actual values helped interpret model behaviour, highlighting issues such as lag, over-smoothing and trend capture.

7. Trading Strategy Integration

Beyond prediction, Week 3 addressed the question of decision-making. A **Sentiment-Weighted Momentum Strategy** was formulated by combining:

- The LSTM's predicted next-day return.
- FinBERT's sentiment score as a confirmation signal.

Trades were executed only when both technical predictions and sentiment direction agreed, reducing risk from noisy or contradictory signals. This step bridged the gap between machine learning outputs and actionable trading logic.

Learning Outcomes and Reflections

Across the three weeks of project, the primary learning outcomes were:

- Developed a foundational understanding of financial market data
- Gained clarity on why sentiment matters in finance
- Learned the strengths and weaknesses of different sentiment analysis approaches
- Understood the importance of domain-specific NLP models
- Appreciated real-world challenges such as data alignment, noise and model assumptions
- Understood why financial data must be treated as a sequence rather than static samples
- Learnt RNNs and LSTMs, including their strengths and limitations
- Built a multimodal time-series model that fuses sentiment and price data